labsheet =06 computer vision Priyanshu Sharma Cu240250980 section=(B) (47)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
# EXPERIMENT 6
# IMAGE SEGMENTATION TECHNIQUES
# ============================================================

print("=" * 65)
print("EXPERIMENT 6")
print("Implementation and Comparative Analysis of")
print("Image Segmentation Techniques")
print("=" * 65)
```

```
=================================================================
EXPERIMENT 6
Implementation and Comparative Analysis of
Image Segmentation Techniques
=================================================================
```

```
# 1. LOAD IMAGE
# ============================================================

# IMPORTANT:
# Put your actual image path here.
# r before the path prevents Windows path errors.

image_path = "/content/pexels-friededia-30649280.jpg"

image = cv2.imread(image_path)

if image is None:
    print("\nERROR: Image not found!")
    print("Please check your image path.")
    print(image_path)
    exit()

print("\nImage loaded successfully!")
print("Image size:", image.shape)


# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
Image loaded successfully!
Image size: (5464, 8192, 3)
```

```
# 2. GRAYSCALE CONVERSION
# ============================================================

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Grayscale conversion completed.")
```

```
Grayscale conversion completed.
```

```
# 3. GAUSSIAN BLUR
# ============================================================

blur = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

print("Gaussian Blur preprocessing completed.")
```

```
Gaussian Blur preprocessing completed.
```

```
# 4. DISPLAY ORIGINAL, GRAYSCALE AND BLURRED IMAGE
# ============================================================
```

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.tight_layout()
plt.show()
```


```
# 5. GLOBAL THRESHOLDING
# ============================================================

print("\n" + "=" * 65)
print("GLOBAL THRESHOLDING")
print("=" * 65)

threshold_value = 127

_, global_threshold = cv2.threshold(
    blur,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

print("Global threshold value:", threshold_value)
```

```
=================================================================
GLOBAL THRESHOLDING
=================================================================
Global threshold value: 127
```

```
# 6. OTSU'S THRESHOLDING
# ============================================================

print("\n" + "=" * 65)
print("OTSU'S THRESHOLDING")
print("=" * 65)

otsu_value, otsu_threshold = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Automatically selected Otsu threshold:", otsu_value)
```

```
=================================================================
OTSU'S THRESHOLDING
=================================================================
Automatically selected Otsu threshold: 147.0
```

```
# 7. ADAPTIVE THRESHOLDING
# ============================================================

print("\n" + "=" * 65)
print("ADAPTIVE THRESHOLDING")
print("=" * 65)

adaptive_threshold = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

print("Adaptive thresholding completed.")
```

```
=================================================================
ADAPTIVE THRESHOLDING
=================================================================
Adaptive thresholding completed.
```

```
# 8. DISPLAY THRESHOLDING RESULTS
# ============================================================

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.tight_layout()
plt.show()
```


```
# 9. WATERSHED SEGMENTATION
# ============================================================

print("\n" + "=" * 65)
print("WATERSHED SEGMENTATION")
print("=" * 65)
print("PRIYANSHU SHARMA CU240250980")

# Use Otsu threshold as starting point
binary = otsu_threshold.copy()

# Noise removal
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
```

```
)

# Sure background
sure_background = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
distance = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Sure foreground
_, sure_foreground = cv2.threshold(
    distance,
    0.5 * distance.max(),
    255,
    0
)

sure_foreground = np.uint8(sure_foreground)

# Unknown region
unknown = cv2.subtract(
    sure_background,
    sure_foreground
)

# Marker labelling
num_labels, markers = cv2.connectedComponents(
    sure_foreground
)

# Add one so background is not 0
markers = markers + 1

# Mark unknown region as 0
markers[unknown == 255] = 0

# Apply watershed
watershed_image = image.copy()

markers = cv2.watershed(
    watershed_image,
    markers
)

# Mark boundaries
watershed_result = image_rgb.copy()

watershed_result[markers == -1] = [255, 0, 0]

print("Watershed segmentation completed.")
print("Number of regions:", num_labels - 1)
print("PRIYANSHU SHARMA CU240250980")
```

```
======================================================================
WATERSHED SEGMENTATION
======================================================================
PRIYANSHU SHARMA CU240250980
Watershed segmentation completed.
Number of regions: 1
PRIYANSHU SHARMA CU240250980
```

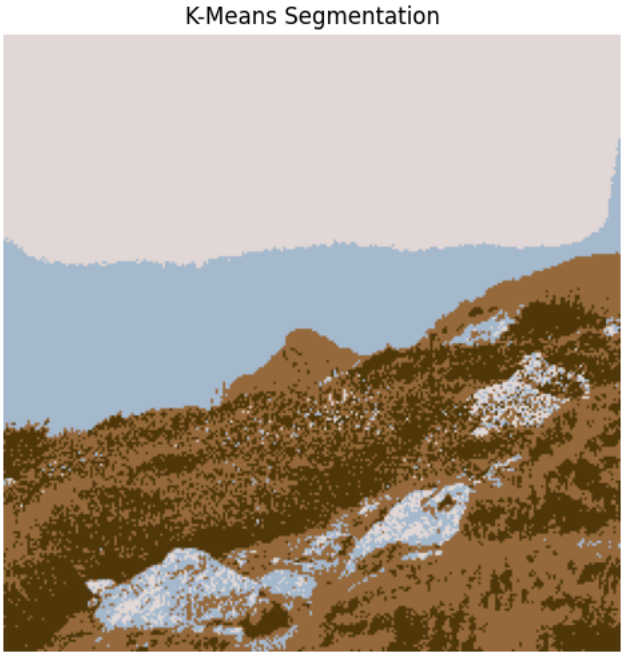
```
# 10. DISPLAY WATERSHED RESULT
# ============================================================

plt.figure(figsize=(10, 6))

plt.imshow(watershed_result)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.show()
```

Watershed Segmentation

```
# 11. K-MEANS CLUSTERING
# ============================================================

print("\n" + "=" * 65)
print("K-MEANS IMAGE SEGMENTATION")
print("=" * 65)

# Resize image to make processing faster
small_image = cv2.resize(
    image_rgb,
    (256, 256)
)

# Convert image into pixel list
pixels = small_image.reshape(
    (-1, 3)
)

# Convert pixels to float
pixels = np.float32(pixels)

# Number of clusters
k = 4

print("Number of clusters:", k)
print("PRIYANSHU SHARMA CU240250980")

# Create KMeans model
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

# Fit model
kmeans.fit(pixels)

# Get cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Get labels
labels = kmeans.labels_

# Replace each pixel with its cluster center
segmented_pixels = centers[labels]

# Reshape back into image
kmeans_result = segmented_pixels.reshape(
    small_image.shape
)

print("K-Means segmentation completed.")
print("PRIYANSHU SHARMA CU240250980")
```

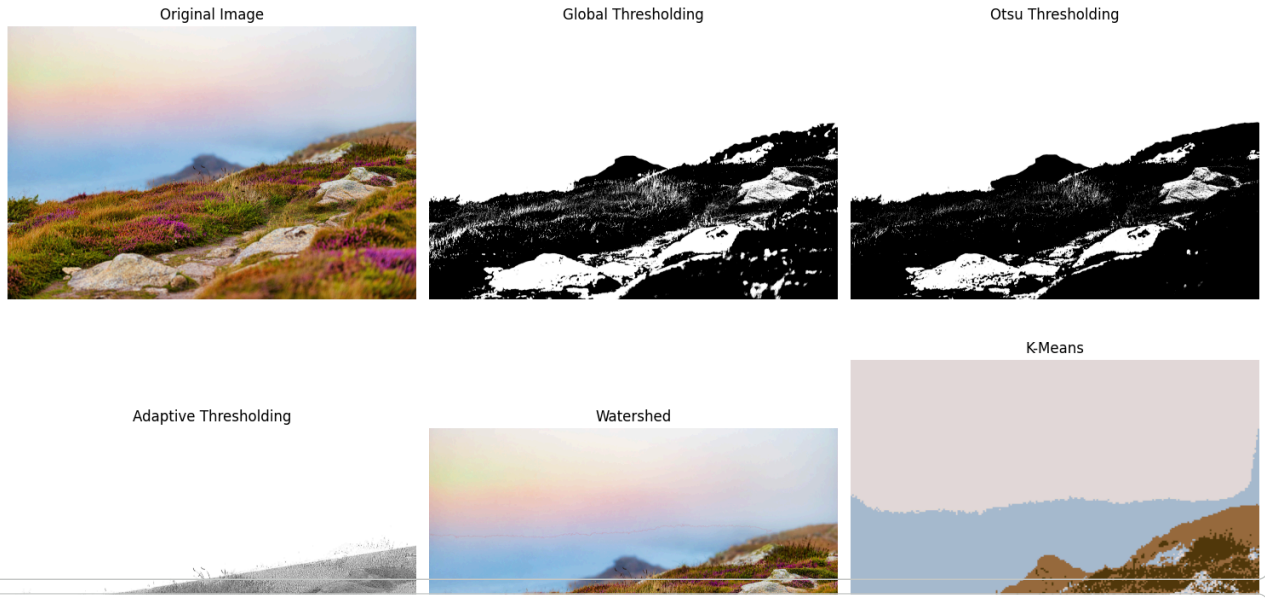
```
=================================================================
K-MEANS IMAGE SEGMENTATION
=================================================================
Number of clusters: 4
PRIYANSHU SHARMA CU240250980
K-Means segmentation completed.
PRIYANSHU SHARMA CU240250980
```

```
# 12. DISPLAY K-MEANS RESULT
# ============================================================

plt.figure(figsize=(10, 6))

plt.imshow(kmeans_result)
plt.title("K-Means Segmentation")
plt.axis("off")

plt.show()
```




```
# 13. FINAL COMPARISON
# ============================================================

print("\n" + "=" * 65)
print("COMPARISON OF SEGMENTATION TECHNIQUES")
print("=" * 65)

plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_result)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(kmeans_result)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
============================================================
COMPARISON OF SEGMENTATION TECHNIQUES
============================================================
```




```
# 14. OBSERVATION
# ============================================================
print("PRIYANSHU SHARMA CU240250980")
print("\n" + "=" * 65)
print("OBSERVATION")
print("=" * 65)

print("""
1. Global Thresholding:
   Separates foreground and background using a fixed threshold.

2. Otsu's Thresholding:
   Automatically selects an optimal threshold value.

3. Adaptive Thresholding:
   Uses local regions and works better when illumination
   is uneven.

4. Watershed:
   Useful for separating touching or overlapping objects.

5. K-Means:
   Groups pixels into different clusters based on their
```